TOKEN OPTIMIZATION, CACHING & BUDGET CONTROLS

AEGIS AGENT | First-person engineering account | Source-grounded implementation

My engineering position. I designed Aegis Agent to make model usage more observable and bounded through lightweight estimation, task-level response caching, and explicit budget checks. I describe the implemented control logic precisely; I do not represent these mechanisms as a verified tokenizer, billing system, or measured reduction in cloud spending.

01 | Where I place the controls

I centralize these functions in core/token_optimizer.py. The TokenOptimizer instance owns its own cache, request history, cumulative token counter, and recent-request timestamps. In core/ai_core.py, I instantiate that optimizer inside AICore and use it in the dispatch and specialist-workflow paths.

Code surface	Verified responsibility
core/token_optimizer.py	Normalization, TTL cache, approximate token estimation, usage recording, budget checks and request-rate signal.
core/ai_core.py	Cache-first dispatch, provider completion on a miss, usage accounting, and sequential specialist workflow.
core/model_router.py	Heuristic model-tier label; not an independent token-billing engine.
core/llm_provider.py	Default offline Echo provider; optional configured HTTP provider.

02 | My request lifecycle

Cache lookup precedes provider completion in AICore.dispatch(). For a hit, I return the cached response with a cached flag and zero tokens reported for that dispatch. For a miss, I resolve a specialist, build a prompt, invoke the configured provider, record approximate usage, and store the response.

The cache-hit field describes that dispatch path; it is not a statement that the original response consumed no tokens. I also distinguish the provider-reported token count returned by dispatch from the separate estimator-based amount recorded by TokenOptimizer.

03 | Token estimation is intentionally approximate

My estimate_tokens() uses the rule max(1, len(text) // 4) for non-null strings and returns zero for None. This is a character-length heuristic. It does not run a provider tokenizer, inspect message framing, or guarantee alignment with an API invoice.

Input to estimator	Result / interpretation
None	0 estimated tokens.
Non-null empty string	1 estimated token because of the minimum.
40-character text	10 estimated tokens.
Task plus response	Two separate estimates are added before the request cap is applied.

04 | My bounded usage accounting

In record_usage(), I estimate task and response lengths, sum them, and cap the recorded amount at max_tokens_per_request. The default cap is 4,000; the default cumulative budget is 100,000. I reject a proposed addition when the cumulative counter plus the capped amount exceeds the budget.

I deliberately distinguish recorded estimated usage from actual provider consumption. Capping the number written to the ledger does not physically prevent a provider from generating more tokens. The provider call in AICore.dispatch() separately passes max_tokens=512; its real enforcement depends on the selected provider implementation.

05 | A critical boundary around the daily budget

Although the parameter is named daily_budget, the implementation shown maintains an in-memory cumulative counter without a date boundary or automatic midnight reset. I therefore document it as an instance-lifetime budget guard rather than a demonstrated calendar-day quota.

The check_budget() method raises PermissionError when the next recorded amount would exceed the threshold. enforce_budget() raises RuntimeError once the accumulated total has reached or exceeded the threshold. These are two distinct checks with different exception types.

06 | My response-cache contract

I normalize a task by trimming it, collapsing whitespace, and lowercasing the result. That normalized string becomes the cache key. A cached entry records the original task, agent name, response, timestamp, and expiry. The default TTL is 300 seconds. Expired entries are discarded when they are looked up.

Behavior	What I can support from source
Equivalent whitespace / case	Tasks can resolve to the same normalized cache key.
TTL expiration	A lookup after expiry removes the entry and returns a miss.
Storage scope	An in-memory dictionary owned by one TokenOptimizer instance.
Cache isolation	The key is task text alone; no tenant, model, provider, or specialist dimension is included.
Eviction policy	TTL-on-lookup exists; no capacity-based LRU or persistent shared cache is shown.

07 | Why I avoid overstating cache safety

I do not claim tenant-safe shared caching merely because other parts of Aegis Agent use tenant identifiers. In this class, _normalize_task(task) is the complete key. If a single optimizer instance serves multiple contexts, identical task text can address the same cached response unless the surrounding architecture supplies an isolation boundary.

I also avoid promising cache correctness across changes in model configuration, prompt templates, or authorization state. The current key does not encode those dimensions. A production-grade shared cache would need an explicit scope and invalidation contract.

08 | A subtle workflow interaction

In AICore.run_workflow(), I iterate over six specialists and collect each agent's handle(task) output. Each iteration records usage and caches a response under the same normalized task key. As a result, later workflow entries can overwrite earlier entries for that task. This method is not six independent provider completions, and the cache does not preserve a per-specialist result set.

09 | Throttling signals versus enforced rate limits

I maintain recent timestamps per specialist and expose throttle_if_needed(agent_name, max_requests_per_minute=30). The function returns a Boolean based on the count in its current recent-request list; it does not sleep, queue, reject, or autonomously throttle requests. I describe it as a rate-limit signal, not a complete enforcement gateway.

The recent-request list is pruned when usage is recorded. A standalone call to throttle_if_needed() does not itself refresh the time window. The accounting also uses a local in-memory history, so it should not be presented as a distributed, multi-worker rate limiter.

10 | Evidence and observability

After recording usage, I attempt to append an llm_usage event through EvidenceLedgerSingleton with the task, model label, estimated tokens, and specialist name. The ledger write is wrapped in a broad exception handler, making it best effort. A successful record_usage() return is not proof that the evidence event was durably persisted.

Observable output	Meaning and limitation
usage_summary()	Cumulative estimated tokens, configured budget, remaining amount, cache-entry count, and request count.
record_usage() return	Estimated amount for the request and remaining in-memory budget.
llm_usage ledger event	Attempted evidence record; failures may be suppressed.
AICore dispatch tokens	Provider-returned total on a miss, or zero reported on a cache hit.

11 | My engineering acceptance boundary

I can substantiate the existence of these controls from the repository source. I do not claim independently measured token savings, benchmarked latency improvements, precise provider billing reconciliation, automatic daily resets, durable distributed quotas, or comprehensive tenant-safe cache isolation. Those outcomes require explicit implementation and executed tests.

Source basis: core/token_optimizer.py, core/ai_core.py, core/model_router.py, and core/llm_provider.py in the public Aegis Agent repository. This document is my implementation account, not an independent certification or a claim that a test suite was executed for this publication.